



CCF-GESP C++四级

编程能力等级认证



第十八课

综 合 练 习



(2023年9月) 变长编码

【问题描述】

小明刚刚学习了三种整数编码方式：原码、反码、补码，并了解到计算机存储整数通常使用补码。但他总是觉得，生活中很少用到 $2^{31}-1$ 这么大的数，生活中常用的0~100这种数也同样需要4个字节的补码表示，太浪费了些。热爱学习的小明通过搜索，发现了一种正整数的变长编码方式。这种编码方式的规则如下：

1. 对于给定的正整数，首先将其表达为二进制形式。例如， $(0)_{\{10\}}=(0)_{\{2\}}$ ， $(926)_{\{10\}}=(1110011110)_{\{2\}}$ 。
2. 将二进制数从低位到高位切分成每组7bit，不足7bit的在高位用0填补。例如， $(0)_{\{2\}}$ 变为0000000的一组， $(1110011110)_{\{2\}}$ 变为0011110和0000111的两组。
3. 由代表低位的组开始，为其加入最高位。如果这组是最后一组，则在最高位填上0，否则在最高位填上1。于是，0的变长编码为00000000一个字节，926的变长编码为10011110和00000111两个字节。

这种编码方式可以用更少的字节表达比较小的数，也可以用很多的字节表达非常大的数。例如，987654321012345678的二进制为 $(0001101\ 1011010\ 0110110\ 1001011\ 1110100\ 0100110\ 1001000\ 0010110\ 1001110)_{\{2\}}$ ，于是它的变长编码为（十六进制表示）CE 96 C8 A6 F4 CB B6 DA 0D，共9个字节。

你能通过编写程序，找到一个正整数的变长编码吗？



(2023年9月) 变长编码

【输入格式】

输入第一行，包含一个正整数N。约定 $0 \leq N \leq 1018$ 。

【输出格式】

输出一行，输出N对应的变长编码的每个字节，每个字节均以2位十六进制表示（其中 A-F 使用大写字母表示），两个字节间以空格分隔。

【输入样例1】

0

【输出样例1】

00

【输入样例2】

926

【输出样例2】

9E 07

【输入样例3】

987654321012345678

【输出样例3】

CE 96 C8 A6 F4 CB B6 DA 0D



数据分析

【输入样例1】

0

【输出样例1】

00

【输入样例2】

926

【输出样例2】

9E 07

【输入样例3】

987654321012345678

【输出样例3】

CE 96 C8 A6 F4 CB B6 DA 0D

原数据	二进制数据	二进制数从低位到高位切分(每组7bit)	由代表低位的组开始, 加入最高位	最终结果(每个字节均以2位十六进制)
0	0	0000000	00000000	00
926	1110011110	0011110 0000111	10011110 0000111 9 E 0 7	9E 07



数据分析

原数据	二进制数据	二进制数从低位到高位切分(每组7bit)	由代表低位的组开始, 加入最高位	最终结果(每个字节均以2位十六进制)
0	0	0000000	00000000	00
926	1110011110	0011110 0000111	10011110 00000111	9E 07

926&0x7f

保留二进制形式的后7位

```
111001110  ----926
& 0111111  ----127
-----
- 0001110  ----30
```

数组{30}

926>>7
111001110 ⇨ 00000111

```
00000111  ----7
& 0111111  ----127
-----
- 0000011  ----7
```

数组{30,7}

7>>7
00000111 ⇨ 00000000



真题解析



数据分析

原数据	二进制数据	二进制数从低位到高位切分(每组7bit)	由代表低位的组开始, 加入最高位	最终结果(每个字节均以2位十六进制)
0	0	0000000	00000000	00
926	1110011110	0011110 0000111	10011110 00000111	9E 07

数组{30,7}

3d0x80

对高位补位1处理

```
00011110 ----30
| 10000000 ----128
-----
- 10011110 ----158
```

数组{30,7}

➡ 数组{158,7}



数据分析

原数据	二进制数据	二进制数从低位到高位切分(每组7bit)	由代表低位的组开始, 加入最高位	最终结果(每个字节均以2位十六进制)
0	0	0000000	00000000	00
926	1110011110	0011110 0000111	10011110 00000111	9E 07

数组{158,7}

158>>4 10011110 ⇨ 00001001 ----9

10011110 & 0x0f ⇨ 00001110 ----14 E

7>>4 00000111 ⇨ 00000000 ----0

00000111 & 0x0f ⇨ 00000111 ----7



真题解析



思路分析

把输入数据的
二进制数从低
位到高位切分



对切分后的数据，
加入最高位



按要求进行最
终输出



把输入数据的二进制数从低位到高位切分

① 输入数字n, 注意数字范围 $0 \leq N \leq 10^{18}$

```
int main() {  
    long long n = 0;  
    cin >> n;  
    int split[10];  
    int l = 0;  
    while(n>0){  
        split[l]=(int)(n & 0x7f);  
        n>>=7;  
        l++;  
    }  
    for(int i=0;i<l-1;i++)  
        split[i]=0x80;  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    return 0;  
}
```



把输入数据的二进制数从低位到高位切分

② 对数字n进行二进制数切分，n大于0时进行切分



```
int main() {  
    long long n = 0;  
    cin >> n;  
    int split[10];  
    int l = 0;  
    while(n>0){  
        split[l]=(int)(n&0x7f);  
        n>>=7;  
        l++;  
    }  
    for(int i=0;i<l-1;i++)  
        split[i]=0x80;  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    return 0;  
}
```

拆分结果保存在int数组



把输入数据的二进制数从低位到高位切分

③ 对数字n进行二进制数切分，n大于0时进行切分

926 & 0x7f

1110011110	----926	00000111	----7
& 01111111	----127	& 01111111	----127
-----		-----	
00011110	----30	00000111	----7
926>>7	数组{30}	7>>7	数组{30,7}
1110011110 ⇨ 00000111		00000111 ⇨ 00000000	

```
int main() {  
    long long n = 0;  
    cin >> n;  
    int split[10];  
    int l = 0;  
    while(n>0){  
        split[l]=(int)(n&0x7f);  
        n>>=7;  
        l++;  
    }  
    for(int i=0;i<l-1;i++)  
        split[i]=0x80;  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    return 0;  
}
```

保留二进制形式的后7位

n的二进制右移7位



对切分后的数据，加入最高位

① 对数组中前 $l-1$ 个元素加入最高位1

```
int main() {  
    long long n = 0;  
    cin >> n;  
    int split[10];  
    int l = 0;  
    while(n>0){  
        split[l]=(int)(n&0x7f);  
        n>>=7;  
        l++;  
    }  
    for(int i=0;i<l-1;i++){  
        split[i]=0x80;  
    }  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    return 0;  
}
```

l-1 结束



对切分后的数据，加入最高位

② 对数组中前*l*-1个元素加入最高位1

数组{30,7} 30 | 0x80

	00011110	----30
	10000000	----128

	10011110	----158

数组{30,7} ⇨ 数组{158,7}



```
int main() {  
    long long n = 0;  
    cin >> n;  
    int split[10];  
    int l = 0;  
    while(n>0){  
        split[l]=(int)(n&0x7f);  
        n>>=7;  
        l++;  
    }  
    for(int i=0;i<l-1;i++){  
        split[i]=0x80;  
        output_code(split[i]);  
    }  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    return 0;  
}
```

异或处理



按要求进行最终输出

① 输出效果为每个字节均以2位十六进制表示（其中 A-F 使用大写字母表示）

10011110 00000111 9E 07



```
void output_digit(int d) {
    if(d>=10)
        cout<<(char)('A'+d-10);
    else
        cout<<(char)('0'+d);
}
void output_code(int s){
    output_digit(s >> 4);
    output_digit(s & 0x0f);
}
int main() {
    ...
    output_code(split[0]);
    for(int i=1;i<l;i++){
        cout << " ";
        output_code(split[i]);
    }
    ...}
```



按要求进行最终输出

② 数组中**每个数字**的二进制数**每四位**对应一个**十六进制数**

10011110 00000111 9E 07



```
void output_digit(int d) {  
    if(d>=10)  
        cout<<(char)('A'+d-10);  
    else  
        cout<<(char)('0'+d);  
}  
void output_code(int s){  
    output_digit(s >> 4);  
    output_digit(s & 0x0f);  
}  
int main() {  
    ...  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    ...}
```

获取前四位

获取后四位



按要求进行最终输出

③ 把每位数字进行输出，注意十六进制数字A-F表示10-15

10011110 00000111 9E 07

数组{158,7}

158>>4 10011110 ⇨ 00001001 ----9

10011110&0x0f⇨ 00001110 ----14 E

7>>4 00000111 ⇨ 00000000 ----0

00000111&0x0f⇨ 00000111 ----7

```
void output_digit(int d) {  
    if(d>=10)  
        cout<<(char)('A'+d-10);  
    else  
        cout<<(char)('0'+d);  
}  
void output_code(int s){  
    output_digit(s >> 4);  
    output_digit(s & 0x0f);  
}  
int main() {  
    ...  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    ...}
```

A-F表示10-15



按要求进行最终输出

⑤ 输出效果：每组数据之间用空格间隔

10011110 00000111 9E 07

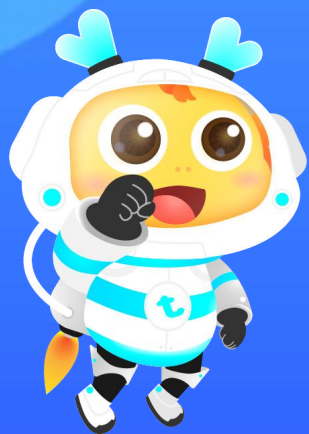
```
void output_digit(int d) {  
    if(d>=10)  
        cout<<(char)('A'+d-10);  
    else  
        cout<<(char)('0'+d);  
}  
void output_code(int s){  
    output_digit(s >> 4);  
    output_digit(s & 0x0f);  
}  
int main() {  
    ...  
    output_code(split[0]);  
    for(int i=1;i<l;i++){  
        cout << " ";  
        output_code(split[i]);  
    }  
    ...  
}
```



【完整代码】

```
#include <bits/stdc++.h>
using namespace std;
void output_digit(int d) {
    if(d>=10)
        cout<<(char)('A'+d-10);
    else
        cout<<(char)('0'+d);
}
void output_code(int s){
    output_digit(s>>4);
    output_digit(s&0x0f);
}
```

```
int main() {
    long long n=0;
    cin>>n;
    int split[10];
    int l = 0;
    while(n>0){
        split[l]=(int)(n&0x7f);
        n>>=7;
        l++;
    }
    for(int i=0;i<l-1;i++) split[i]=0x80;
    output_code(split[0]);
    for(int i=1;i<l;i++){
        cout << " ";
        output_code(split[i]);
    }
    return 0;
}
```



感谢你的学习